

NoteHub要件定義書

概要

NoteHubは複数ユーザーが同一ドキュメントを同時に開き、リアルタイムで共同作業で編集できるオンラインエディタ

ユーザーはアカウントの登録、ログイン後にワークスペースを作成し、その中でドキュメントを作成、編集、削除できる

目的

チーム単位での作業をリアルタイムで可視化しながら、共同作業できる環境を提供する

ユーザーへの解決したい課題と内容

課題	内容
編集内容が同期されず二重管理になる	最新更新の連携にタイムラグがあり、一つのタスクを二重で実施してしまうリスク
自動保存	自動保存
過去の状態に戻せない	誤削除、変更した内容を復元する手段がなく、やり直しになるリスク

実装する機能

機能	内容
ユーザー登録・ログイン	Email/Passwordによるアカウント登録、ログイン、ログアウト
ワークスペース管理	ワークスペースの作成・編集・削除、メンバーの招待・削除
ドキュメント管理	ドキュメントの作成・編集・削除、検索
リアルタイム同時編集	WebSocketを利用し複数ユーザー間での編集内容のリアルタイム編集
テキスト編集	Monaco Editorによるコード・文章の編集

編集者表示	現在ドキュメントを開いている・編集しているユーザーをアイコン等で表示
自動保存	一定間隔・一定操作ごとにドキュメント内容を自動的に保存
編集履歴・バージョン管理	過去のバージョンの一覧表示、任意バージョンへの復元

User

できること	内容
ワークスペースの作成	自分をホストのワークスペースを作成できる
ワークスペースへの参加	ホストから招待されたワークスペースに参加できる
ドキュメントの作成・編集・削除	参加しているワークスペース内のドキュメントに対して操作できる
共同編集	同じドキュメントを開いた他メンバーとリアルタイムに編集を共有できる

Host

できること	内容
メンバー管理	メンバーの招待・削除
ワークスペース管理	ワークスペースの作成・削除

リアルタイム共同編集

項目	内容	補足
通信方式	Gorilla WebSocket	クライアント⇄サーバー間
変更内容反映	WebSocketサーバーは編集内容をRedisに送ってほかのサーバー（メンバー）にも知らせる	ント単位のチャンネル配信複数サーバーインスタンス間でも同期可能

同期単位	ドキュメント単位	document_id) 同一ドキュメントを開いたユーザーのみ
再接続	ネットワークが切れたりサーバーとの接続が切断された場合でも、一定間隔で再接続を試行し編集内容を正しく復元する	再接続時は最新の保存内容を再取得し反映

自動保存

項目	内容
保存タイミング	編集操作から一定時間経過後（60秒）
保存先	PostgreSQL
保存者	ブラウザではなく、サーバーが保存処理を行う
失敗時	保存失敗時はリトライし、クライアントへ保存ステータスを通知する
保持	最新の状態のみ

編集履歴・バージョン管理

項目	内容
復元用の保存タイミング	自動保存とは別に一定期間経過ごとにバージョンとして保存（10分ぐらい？）
保存内容	ドキュメント本文、更新者、更新日時
復元	任意のバージョンを選択し、現在のドキュメントへ復元できる
保持	過去の状態を履歴として残す

認証・セキュリティ

項目	内容
登録	Email／Password
AccessToken	JWT 有効期限を短く設定 メモリ上（Reactの状態管理や変数）だけで保持（ページリロードで消える）
パスワード	ハッシュ化（bcrypt）して保存し、平文は保持しない
WebSocket認証	本当にログイン済みのユーザーか確認（WebSocket開始前に）

操作権限	ワークスペース・ドキュメントへのアクセスは、所属メンバーかどうかをAPI・WebSocketで確認
------	---

ユーザー登録

ユーザーはEmail、パスワード、表示名を入力してアカウントを登録

- Emailはシステム内で一意（唯一）
- Emailの形式を確認する（@が入っているかなど）
- パスワードは所定の文字数・複雑を満たしているか
- パスワードはbcryptでハッシュ化して保存
- パスワードの平文をデータベース、ログ、レスポンスへ出力しない
- 登録済みEmailによる重複登録禁止

パスワード要件

- 8文字以上
- 英字と数字を含める
- 最大文字数を設定し、過剰に長い入力を禁止（15文字）
- ログイン失敗時に、Email、パスワードのどちらが間違っていることを特定できるメッセージを返さない

ログイン処理内容

- Emailに一致するユーザーを取得
- 削除済みまたは無効化されたユーザーでないことを確認
- bcryptを使用してパスワードを照合
- 認証成功後にAccessTokenを発行
- ログイン成功、失敗をログへ記録

AccessToken

API認証

WebSocket認証

ワークスペース権限

ドキュメント権限

レート制限（Token Bucket方式）

タブ開きすぎない

API、クライアントのペア 同じアカウントから何個まで別媒体でログインできるか

ログイン試行制限

WebSocket編集イベント制限

通信セキュリティ

リフレッシュトークン

ログ

cookieで文字データ保存

30分に一回、ページ移動、タブ消したときにcookieからDBに移動させて保存

ローカルストレージには絶対しない

cookie保存したらcookie削除したらデータ消えちゃう

パスワード同じぐらい秘匿性高い

Redie単体サーバーを作ってそこに保存していく